

Proyecto de Business Intelligence: Dashboard de Ventas TechCore

Descripción del Proyecto

Este repositorio documenta el **proyecto integral de Business Intelligence** para **TechCore**, una cadena líder en tecnología. El proyecto abarca el ciclo de vida completo del análisis de datos, comenzando con la **limpieza y transformación de datos** (Avance_1...), seguido por el **modelado relacional en Python** (Avance_2...), y culminando en el desarrollo de un **dashboard interactivo de 9 páginas en Power BI** (Avance_3...).

El objetivo final es proporcionar una herramienta de análisis **robusta, escalable y segura** que permita a los diferentes niveles gerenciales (Nacional, Regional y Sucursal) tomar decisiones estratégicas basadas en datos.

Estructura del Proyecto

La estructura de archivos refleja las tres fases principales del proyecto, desde los datos crudos hasta el informe final.

```
Proyecto_TechCore/
├── 1. Datos Crudos (Dataset Original)
│   └── ventas.csv
├── 2. Fase de Limpieza (Power Query / Python)
│   ├── Avance_1_Limpieza_Transformacion.pbix
│   └── ventasTransformed.csv
├── 3. Fase de Modelado (Python)
│   ├── Avance_2_Modelo_Relacional.ipynb
│   └── modeloVentas.xlsx (Salida con 8 tablas de modelo)
└── 4. Dashboard Final (Power BI)
    └── Avance_3_Dashboard_PowerBI.pbix
```

Proceso de Construcción del Dashboard

Este dashboard es el resultado de un proceso iterativo de construcción y depuración.

Fase 1: Carga y Modelado de Datos (Esquema de Estrella)

Ingesta de Datos

Se cargaron las 8 tablas de negocio (Ciudades, Clientes, DetalleFacturas, etc.) desde los archivos CSV generados por el Avance 2. Se ignoraron explícitamente las tablas de análisis (MarcasMasVendidas,

ProductosMasVendidos) ya que calculamos esa información dinámicamente en el dashboard.

Modelado (Vista de Modelo)

Se organizaron las tablas en un **Esquema de Estrella** para optimizar el rendimiento:

- **Tablas de Hechos** (Centro): **Facturas** y **DetalleFacturas**
- **Tablas de Dimensiones** (Bordes): **Cientes**, **Productos**, **Sucursales**, **Ciudades**, **Vendedores**, **MetodosPago**
- **Relaciones**: Se establecieron relaciones 1-a-muchos (ej. **Productos[ProductoID]** → **DetalleFacturas[ProductoID]**)

Buenas Prácticas

Se ocultaron todas las claves foráneas (ej. **Facturas[CienteID]**) de la vista de informe para simplificar la interfaz y forzar el uso de las dimensiones correctas.

Fase 2: Enriquecimiento del Modelo (Columnas y Tablas DAX)

Tabla de Calendario (dCalendario)

Se creó una tabla de fechas dedicada usando **CALENDARAUTO()** y se enriqueció con columnas como **Año**, **NombreMes** y **AñoMes** para la inteligencia de tiempo. Se marcó como tabla de fechas oficial.

Jerarquía de Ubicación (Paso Clave)

Problema: **NombreCiudad** y **SucursalNombre** estaban en tablas separadas, impidiendo una jerarquía.

Solución: Se creó una columna calculada en la tabla **Sucursales** para "traer" el nombre de la ciudad:

```
NombreCiudad_Rel = RELATED(Ciudades[NombreCiudad])
```

Esto permitió crear la jerarquía **Jerarquía de Ubicación** (**NombreCiudad_Rel** > **SucursalNombre**) dentro de la tabla **Sucursales**.

Columna de Agrupación (Rango de Edad)

Se creó una columna calculada en **Cientes** usando **SWITCH(TRUE(), ...)** para agrupar las edades y facilitar la segmentación.

Fase 3: Creación y Depuración de Medidas DAX (El Cerebro)

Esta fue la fase más crítica donde iteramos para encontrar la lógica de negocio correcta.

● Arreglando VentasTotales (La "Fuente Única de Verdad")

Problema: Nuestro primer intento, `SUM(Facturas[TotalVenta])`, fallaba. Un gráfico de "Ventas por Marca" (Página 3) mostraba todas las barras del mismo tamaño.

Diagnóstico: El filtro de la dimensión `Productos` "baja" por la relación a `DetalleFacturas`, pero no puede "subir" contra la relación para filtrar `Facturas`.

Solución: Se estableció como medida base la suma del subtotal en la tabla de detalles, que SÍ recibe el filtro. Esta decisión arregló todos los gráficos de producto.

```
VentasTotales = SUM(DetalleFacturas[Subtotal])
```

● Arreglando % Margen (El error del 1,01 %)

Problema: Nuestra medida `% Margen (Simulado)` mostraba **1,01 %** en el total, pero **35,00 %** al seleccionar un filtro.

Diagnóstico: Había una inconsistencia de medidas. `[VentasTotales]` sumaba `DetalleFacturas[Subtotal]`, mientras que `[CostoTotal (Simulado)]` intentaba recalcular el costo usando `SUMX` y `RELATED(Productos[PrecioUnitario])`. Los datos de origen no coincidían y causaban el error.

Solución: Se redefinió el costo para que dependiera directamente de la medida de ventas ya validada. Esto forzó la consistencia y estabilizó el margen en **35,00 %** en todos los contextos.

```
CostoTotal (Simulado) = [VentasTotales] * 0.65
% Margen (Simulado) = DIVIDE( ([VentasTotales] - [CostoTotal (Simulado)]),
[VentasTotales], 0 )
```

Fase 4: Diseño de Visualizaciones (9 Páginas)

Se construyeron **9 páginas**, cada una con KPIs y 4+ visualizaciones. Durante este proceso, se tomaron decisiones clave de visualización:

Filtros Top N (Pág. 4 y 5)

Para los gráficos "Top 20 Clientes" y "Top 10 Vendedores", se usó el panel de **Filtros**. Se configuró el filtro **N principal** (ej. Superior 20) y se arrastró la medida `[VentasTotales]` al campo **Por valor** para definir el ranking.

Reemplazo de Histograma (Pág. 7)

El histograma de `TotalVenta` estaba "aplastado" por datos atípicos, haciéndolo ilegible. Se reemplazó por un gráfico mucho más claro: **TicketPromedio por MetodoPago**, que reveló el valioso insight de que los pagos digitales tienen un ticket promedio más alto.

Reemplazo de KPI Interactivo (Pág. 2)

Nuestro intento de crear un KPI de "Ranking de Ciudad" fracasó (**ISINSCOPE**, **HASONEVALUE**, etc.) debido a que las interacciones del filtro no se aplicaban. Se reemplazó por un KPI más simple e infalible:

```
Total Sucursales = DISTINCTCOUNT(Sucursales[SucursalID])
```

Este KPI reacciona correctamente a los filtros.



Implementación de Seguridad (Row-Level Security)

Esta fue la fase final del proyecto. Se implementó una **RLS dinámica** para filtrar la vista del informe según quién inicie sesión.

El Desafío

Crear 3 roles con diferentes niveles de acceso:

- **Gerente Nacional:** Ve todos los datos de todas las ciudades
- **Gerente Regional:** Ve únicamente los datos de su ciudad asignada
- **Gerente Sucursal:** Ve únicamente los datos de su sucursal asignada

La Implementación (Paso a Paso)

Paso 1: Crear la Tabla de Usuarios (UsuariosRLS)

Se usó **Inicio > Especificar datos** para crear una tabla manual con los correos, roles y valores de filtro (ej. "Bogotá" o "TechCore Cali").

Paso 2: ¡Paso Crítico de Limpieza!

Problema: Al pegar los datos, los títulos (**UserEmail** (**UPN**), **RolAsignado**) se pegaron como la **Fila 1**, pero los nombres de columna reales eran **Columna1**, **Columna2**. Esto causaba que las reglas DAX fallaran.

Solución: Se fue a **Transformar datos** (Power Query), se seleccionó la tabla **UsuariosRLS** y se usó la función **Usar la primera fila como encabezado**.

Paso 3: Aislar la Tabla de Seguridad

En la **Vista de Modelo**, nos aseguramos de que la tabla **UsuariosRLS** **NO** estuviera conectada a ninguna otra tabla. Esto es vital. La RLS funciona filtrando las tablas de dimensiones, no conectándose a la tabla de seguridad.

Paso 4: Crear los Roles y Reglas DAX

Se usó **Modelado > Administrar roles** para crear los 3 roles.

La Lógica: ¿Cómo Funciona Cada Regla?

Rol 1: Gerente Nacional

Regla: *(Ninguna)*

Explicación: Al no tener ninguna regla DAX, este rol no tiene filtros aplicados y puede ver todos los datos de la compañía.

Rol 2: Gerente Regional

Regla (Aplicada sobre la tabla **Ciudades**):

```
[NombreCiudad] =  
LOOKUPVALUE(  
    UsuariosRLS[CiudadFiltro],  
    UsuariosRLS[UserEmail (UPN)], USERPRINCIPALNAME(),  
    UsuariosRLS[RolAsignado], "Gerente Regional"  
)
```

Explicación (Paso a Paso):

1. **USERPRINCIPALNAME()**: Esta función obtiene el correo del usuario que ha iniciado sesión (ej. **gerente.bogota@techcore.com**)
 2. **LOOKUPVALUE(...)**: Esta función actúa como un "BuscarV" y dice: "En la tabla **UsuariosRLS**, encuéntrame el valor de la columna **[CiudadFiltro]**...
 - ...donde el **[UserEmail (UPN)]** sea igual al del usuario actual (ej. **gerente.bogota@techcore.com**)
 - ...Y donde el **[RolAsignado]** sea '**Gerente Regional**'"
 3. El resultado de **LOOKUPVALUE** es **Bogotá**
 4. **[NombreCiudad] = "Bogotá"**: La regla final filtra la tabla **Ciudades** para que solo muestre la fila "Bogotá"
 5. Este filtro se propaga "hacia abajo" por el modelo (a **Sucursales**, luego a **Facturas**, luego a **DetalleFacturas**), ocultando todos los datos que no sean de Bogotá
-

Rol 3: Gerente Sucursal

Regla (Aplicada sobre la tabla **Sucursales**):

```
[SucursalNombre] =  
LOOKUPVALUE(  
    UsuariosRLS[SucursalFiltro],
```

```
UsuariosRLS[UserEmail (UPN)], USERPRINCIPALNAME(),  
UsuariosRLS[RolAsignado], "Gerente Sucursal"  
)
```

Explicación (Paso a Paso):

1. **USERPRINCIPALNAME()**: Obtiene el correo del usuario (ej. `gerente.suc.cali@techcore.com`)
2. **LOOKUPVALUE(...)**: Busca en **UsuariosRLS** el valor de **[SucursalFiltro]**...
 - ...donde el **[UserEmail (UPN)]** sea `gerente.suc.cali@techcore.com`
 - ...Y donde el **[RolAsignado]** sea `'Gerente Sucursal'`
3. El resultado de **LOOKUPVALUE** es **TechCore Cali**
4. **[SucursalNombre] = "TechCore Cali"**: La regla filtra la tabla **Sucursales** a esa única fila, y el filtro se propaga por todo el modelo

Paso 5: Pruebas de Seguridad

Se usó **Modelado > Ver como** para probar cada escenario:

Prueba A: Gerente Regional

- Se seleccionó **Gerente Regional Y Otro usuario**, escribiendo `gerente.bogota@techcore.com`
- **Resultado:** ☒ Éxito. El dashboard mostró una barra amarilla de confirmación y filtró todos los datos solo a Bogotá

Prueba B: Gerente Sucursal

- Se seleccionó **Gerente Sucursal Y Otro usuario**, escribiendo `gerente.suc.cali@techcore.com`
- **Resultado:** ☒ Éxito. El dashboard se filtró para mostrar únicamente la sucursal "TechCore Cali"

Conclusión

Este proyecto demuestra la implementación completa de una solución de Business Intelligence empresarial, desde la limpieza de datos hasta la seguridad a nivel de fila. Las lecciones aprendidas durante el proceso de depuración y las decisiones técnicas documentadas sirven como guía para futuros proyectos de BI.

© 2025 TechCore Analytics Team